

PRACTICAL FILE

INTRODUCTION TO ALGORITHMS AND ANALYSIS

Name: Dhruv Dilipbhai Solanki

Enrollment No: 202403103510067

Class / Semester: 5-D

Subject: INTRODUCTION TO ALGORITHMS AND ANALYSIS

Institute: AMTICS

Index

Sr. No.	Practical Title	Date	Signature
1	Perform a practical and implement Merge Sort using divide-and-conquer and analyze its time complexity in best, worst, and average cases.		
2	To implement Quicksort and Heap Sort, compare their performance and discuss sorting decision trees for complexity analysis.		
3	To implement Counting Sort or Radix Sort and observe how non-comparison-based sorting achieves linear complexity under certain conditions.		
4	To implement QuickSelect algorithm to find the k-th smallest element in an unsorted array efficiently.		
5	To implement hashing using open addressing with linear and quadratic probing, and analyze collision resolution techniques.		
6	To implement insertion, deletion, and traversal in a Binary Search Tree and analyze its time complexity.		
7	To implement insertion operation in a Red-Black Tree and observe how balancing is maintained after each insertion.		
8	To implement Breadth-First Search (BFS) and Depth-First Search (DFS) on a graph and compare their traversal order and applications.		
9	To implement Dijkstra's algorithm for finding the shortest path from a single source node to all other nodes in a weighted graph.		
10	To implement the 0/1 Knapsack problem using dynamic programming and demonstrate optimal substructure and overlapping subproblems.		

PRACTICAL 1

AIM: Perform a practical and implement Merge Sort using divide-and-conquer and analyze its time complexity in best, worst, and average cases.

Program Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// C program for the implementation of merge sort
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int leftArr[n1], rightArr[n2];

    // Copy data to temporary arrays
    for (i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        rightArr[j] = arr[mid + 1 + j];

    // Merge the temporary arrays back into arr[left..right]
    i = 0;
    j = 0;
    k = left;
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        }
```

```
        }
        else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of leftArr[], if any
    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }

    // Copy the remaining elements of rightArr[], if any
    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}

// The subarray to be sorted is in the index range [left-right]
void mergeSort(int arr[], int left, int right) {
    if (left < right) {

        // Calculate the midpoint
        int mid = left + (right - left) / 2;

        // Sort first and second halves
```

```
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

int main() {
    int count = 1000000;
    int marr[count];
    clock_t start, end;

    for (int i = 0 ; i < count; i++) {
        marr[i] = rand() % count;
    }

    start = clock();
    // Sorting arr using mergesort
    mergeSort(marr, 0, count - 1);
    end = clock();

    long double passedTime = (end - start) / (long double)
CLOCKS_PER_SEC;

    printf("Merge Sort Performance for %d length: %Lf second\n",
count, passedTime);
    return 0;
}
```

Output Screenshots:

Merge Sort Performance for 1000000 length: 0.226483 second

PRACTICAL 2

AIM: To implement Quicksort and Heap Sort, compare their performance and discuss sorting decision trees for complexity analysis.

Program Code:

```
#include <stdio.h>
#include <sys/time.h>
#include <stdlib.h>
#include <time.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    // Initialize pivot to be the first element
    int p = arr[low];
    int i = low;
    int j = high;

    while (i < j) {
        // Find the first element greater than
        // the pivot (from starting)
        while (arr[i] <= p && i <= high - 1) {
            i++;
        }

        // Find the first element smaller than
        // the pivot (from last)
        while (arr[j] > p && j >= low + 1) {
            j--;
        }
        if (i < j) {
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[low], &arr[j]);
    return j;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
```

```
        // call partition function to find Partition Index
        int pi = partition(arr, low, high);

        // Recursively call quickSort() for left and right
        // half based on Partition Index
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
// To heapify a subtree rooted with node i
void heapify(int arr[], int n, int i)
{
    // Initialize largest as root
    int largest = i;

    // left index = 2*i + 1
    int l = 2 * i + 1;

    // right index = 2*i + 2
    int r = 2 * i + 2;

    // If left child is larger than root
    if (l < n && arr[l] > arr[largest])
        largest = l;

    // If right child is larger than largest so far
    if (r < n && arr[r] > arr[largest])
        largest = r;

    // If largest is not root
    if (largest != i)
    {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}
```

```
// Main function to do heap sort
void heapSort(int arr[], int n)
{

```

```
// Build heap (rearrange vector)
for (int i = n / 2 - 1; i >= 0; i--)
    heapify(arr, n, i);

// One by one extract an element from heap
for (int i = n - 1; i > 0; i--)
{
    // Move current root to end
    int temp = arr[0];
    arr[0] = arr[i];
    arr[i] = temp;

    // Call max heapify on the reduced heap
    heapify(arr, i, 0);
}

int main() {
    clock_t start, end;

    srand(time(NULL));

    int count = 1000000;
    int qarr[count];
    int harr[count];

    for (int i = 0 ; i < count; i++) {
        qarr[i] = rand() % count;
    }

    for (int i = 0 ; i < count; i++) {
        harr[i] = rand() % count;
    }

    start = clock();
    // calling quickSort() to sort the given array
    quickSort(qarr, 0, count - 1);
    end = clock();

    long double passedTimeq = (end - start) / (long double)
CLOCKS_PER_SEC;

    start = clock();
    heapSort(harr, count);
    end = clock();
}
```

```
        long double passedTimeh = (end - start) / (long double)
CLOCKS_PER_SEC;

        printf("Quick Sort Performance for %d length: %Lf second\n",
count, passedTimeq);
        printf("Heap Sort Performance for %d length: %Lf second\n", count,
passedTimeh);

        return 0;
}
```

Output Screenshots:

```
Quick Sort Performance for 1000000 length: 0.169758 second
Heap Sort Performance for 1000000 length: 0.413975 second
```

PRACTICAL 3

AIM: To implement Counting Sort or Radix Sort and observe how non-comparison-based sorting achieves linear complexity under certain conditions.

Program Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void countsort(int arr[], int n) {
    int maxval = 0;

    // Find the maximum element
    for (int i = 0; i < n; i++) {
        if (arr[i] > maxval) {
            maxval = arr[i];
        }
    }

    // Create and initialize cntArr array
    int* cntArr = (int*)calloc(maxval + 1, sizeof(int));

    // Count frequency of each element
    for (int i = 0; i < n; i++) {
        cntArr[arr[i]]++;
    }

    // Compute prefix sum
    for (int i = 1; i <= maxval; i++) {
        cntArr[i] += cntArr[i - 1];
    }

    // Build output array
    int* ans = (int*)malloc(n * sizeof(int));
    for (int i = n - 1; i >= 0; i--) {
        ans[cntArr[arr[i]] - 1] = arr[i];
        cntArr[arr[i]]--;
    }

    // Copy sorted elements back to arr[]
    for (int i = 0; i < n; i++) {
        arr[i] = ans[i];
    }

    // Free dynamically allocated memory
    free(cntArr);
    free(ans);
}

// Driver code
```

```
int main() {
    clock_t start, end;

    int count = 1000000;
    int carr[count];
    srand(time(NULL));

    for (int i = 0 ; i < count; i++) {
        carr[i] = rand() % count;
    }

    // Sorting the array
    start = clock();
    countsort(carr, count - 1);
    end = clock();

    long double passedTimec = (end - start) / (long double)
CLOCKS_PER_SEC;

    printf("Counting Sort Performance for %d length: %Lf second\n",
count, passedTimec);

    // Printing the sorted array

    return 0;
}
```

Output Screenshots:

Counting Sort Performance for 1000000 length: 0.053311 second

PRACTICAL 4

AIM: To implement QuickSelect algorithm to find the k-th smallest element in an unsorted array efficiently.

Program Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <time.h>

// Function to swap two elements
void swap(int *a, int *b);

int partition(int arr[], int l, int r) {
    int x = arr[r], i = l;
    for (int j = l; j <= r - 1; j++) {
        if (arr[j] <= x) {
            swap(&arr[i], &arr[j]);
            i++;
        }
    }
    swap(&arr[i], &arr[r]);
    return i;
}

int kthSmallest(int arr[], int l, int r, int k) {
    if (k > 0 && k <= r - l + 1) {
        int index = partition(arr, l, r);

        // If position is the same as k.
        if (index - l == k - 1)
            return arr[index];

        // If position is more, recur for the left subarray.
        if (index - l > k - 1)
            return kthSmallest(arr, l, index - 1, k);

        // Else recur for the right subarray.
        return kthSmallest(arr, index + 1, r,
                           k - index + l - 1);
    }

    return INT_MAX;
}
```

```
}

// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Driver program to test the above methods.
int main() {
    int count = 1000000;
    int qarr[count];
    clock_t start, end;

    for (int i = 0 ; i < count; i++) {
        qarr[i] = rand() % count;
    }

    int k = 74;
    qarr[rand() % count] = k;

    start = clock();
    kthSmallest(qarr, 0, count - 1, k);
    end = clock();

    long double passedTimeq = (end - start) / (long double)
CLOCKS_PER_SEC;

    printf("Quick Select Performance for %d length: %Lf second\n",
count, passedTimeq);
    return 0;
}
```

Output Screenshots:

Quick Select Performance for 1000000 length: 0.015469 second

PRACTICAL 5

AIM: To implement hashing using open addressing with linear and quadratic probing, and analyze collision resolution techniques.

Program Code:

```
#include <stdio.h>
#define SIZE 10
void initialize(int table[])
{
    for (int i = 0; i < SIZE; i++)
    {
        table[i] = -1;
    }
}
void linearProbing(int table[], int key)
{
    int index = key % SIZE;

    for (int i = 0; i < SIZE; i++)
    {
        int position = (index + i) % SIZE;

        if (table[position] == -1)
        {
            table[position] = key;
            return;
        }
    }
    printf("Hash table is full. Cannot insert %d\n", key);
}
void quadraticProbing(int table[], int key)
{
    int index = key % SIZE;
    for (int i = 0; i < SIZE; i++)
    {
        int position = (index + i * i) % SIZE;
        if (table[position] == -1)
        {
            table[position] = key;
            return;
        }
    }
    printf("Hash table is full. Cannot insert %d\n", key);
}

void display(int table[])
```

```
{
    for (int i = 0; i < SIZE; i++)
    {
        if (table[i] == -1)
        {
            printf("Index %d: Empty\n", i);
        }
        else
        {
            printf("Index %d: %d\n", i, table[i]);
        }
    }
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int keys[n];
    int linearTable[SIZE];
    int quadraticTable[SIZE];

    initialize(linearTable);
    initialize(quadraticTable);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &keys[i]);
    }

    for (int i = 0; i < n; i++)
    {
        linearProbing(linearTable, keys[i]);
        quadraticProbing(quadraticTable, keys[i]);
    }
    printf("\nHash Table using Linear Probing:\n");
    display(linearTable);

    printf("\nHash Table using Quadratic Probing:\n");
    display(quadraticTable);

    return 0;
}
```

Output Screenshots:

Enter number of elements:

5

Enter elements:

13

63

12

64

23

Hash Table using Linear Probing:

Index 0: Empty

Index 1: Empty

Index 2: 12

Index 3: 13

Index 4: 63

Index 5: 64

Index 6: 23

Index 7: Empty

Index 8: Empty

Index 9: Empty

Hash Table using Quadratic Probing:

Index 0: Empty

Index 1: Empty

Index 2: 12

Index 3: 13

Index 4: 63

Index 5: 64

Index 6: Empty

Index 7: 23

Index 8: Empty

Index 9: Empty